CUHK(SZ) Food Ordering System Report

1 OVERALL PROJECT DESCRIPTION

The project is a CLI food-ordering system inspired by CUHK-Shenzhen's food-ordering mini-programme, Fanfou. It simulates the full workflow of ordering, delivery assignment, and admin operational management.

The architecture consists of the following layers:

- Frontend: Python CLI with standardised UI (tables, banners, colours, emojis)
- Backend: Flask REST API that performs input validation, authentication, and SQL operations
- Database: SQLite relational database

The system supports two roles (users and admins), each with distinct functions. It can handle store browsing, menu viewing, ordering, order modifications, driver management, store/menu management, and order tracking.

2 REQUIREMENTS ANALYSIS

The functions of the two roles (users and admins) are as follows:

Users can:

1. Register, log in and log out
2. Browse stores,
    a. View store menus
    b. Place new orders (with the option to have a delivery address that differs from the default address)
3. View their order history
    a. View a specific order's breakdown
    b. Modify unshipped orders
        i. Modify delivery address
        ii. Cancel the order
    c. Confirm an order (mark as completed)
4. Update personal account information

Admins can:

1. Log in and log out
2. Manage unshipped orders
    a. Assign a driver
    b. Mark the order as shipped
3. Manage delivery staff information
    a. Set a driver's status as active or inactive
    b. Edit a driver's information
    c. Add a new driver
4. Manage store information
    a. Manage the menu information of a store (add, edit, delete menu items)
    b. Add a new store
    c. Edit store information # E.g. Haroma to Shampoo, 8th College to Duan Family
    d. Delete store # Store contracts may expire
5. View combined order history (filter by status, stores, and users if needed)
6. Update account information

3 DATABASE AND SQL DESIGN

Based on the requirements, the necessary entities are: Account, Driver, MenuItem, OrderItem, Orders, and Store.

The relationships are:

- One Account -> many Orders
- One Driver -> many Orders
- One MenuItem -> many OrderItems
- One Order -> one Account
- One Order -> one Store
- One Order -> one Driver (optional)
- One Store -> many MenuItems

The schema for each table is as follows:

- Account (AccountID PK, Username, Password, Role, Name, Address, Phone)
- Driver (DriverID PK, Name, Phone, IsActive (0,1))
- MenuItem (MenuItemID PK, StoreID FK -> Store, Name, Price)
- OrderItem (OrderItemID PK, OrderID FK -> Orders, MenuItemID FK -> MenuItem, Quantity, UnitPrice)
- Orders (OrderID PK, StoreID FK -> Store, UserID FK -> Account, Status (UNSHIPPED/SHIPPED, COMPLETED), DeliveryAddress, DriverID FK NULL, OrderTime)
- Store (StoreID PK, Name, Location)

Unlike MenuItem, OrderItem is directly affiliated with a specific Orders entry created by a user. It stores UnitPrice to preserve the historical price even if the menu changes later. Each order includes an OrderTime timestamp, which is set in local time for more insightful order histories. Moreover, the DeliveryAddress of Orders may override the default address for an order without changing the original Account.Address. The account address may only be updated using the actual account information modification function.

In the initial database population, there are:

- 40 sample accounts (38 users and 2 admins)
- 42 menu items
- 7 stores (referenced from CUHK-Shenzhen canteens)
- 10 delivery staff

The Orders and OrderItem tables do not have entries at the beginning because users have not yet created any orders.

4 BACK- AND FRONT-END DESIGN AND IMPLEMENTATION

4.1 Back-end

The back-end API (Flask) design is as follows:

User APIs	Admin APIs
• POST /register • POST /login • GET /users/<user_id> • GET /users/<user_id>/orders	• GET /admin/orders/unshipped • POST /admin/orders/<order_id>/assign • POST /admin/orders/<order_id>/ship

• POST /users/<user_id>/edit • GET /orders/<order_id> • POST /orders/<order_id>/confirm • POST /orders/<order_id>/edit_address • POST /orders/<order_id>/cancel • GET /stores • GET /menu/<store_id> • POST /orders # user places an order	• GET /admin/drivers • GET /admin/drivers/active • POST /admin/drivers/set_active • POST /admin/drivers/create • POST /admin/drivers/<driver_id>/create • POST /admin/drivers/<driver_id>/edit • GET /admin/stores • POST /admin/stores/create • POST /admin/stores/<store_id>/edit • POST /admin/stores/<store_id>/delete • GET /admin/stores/<store_id>/menu • POST /admin/stores/<store_id>/menu/add • POST /admin/menu/<item_id>/edit • POST /admin/menu/<item_id>/delete • GET /admin/orders (with filters)

Each route will receive a JSON, validate it, and produce error messages when necessary. It will then perform SQL operations on the database and return a JSON response to the front-end. Passwords are hashed using bycript in the back-end.

4.2 Front-end

Once the user or admin logs in, they will be directed to different main menus, where they can interact with various functions. The main menu displays choices coded with numbers. For choices or actions deeper in the programme, the user is sometimes prompted to input a letter representing the action, such as A for modifying (A)ddress and C for (C)ancelling an order. The main menu will loop unless the user purposely selects the log-out option.

The front-end code is split into specific functions according to those identified in the requirements analysis for better clarity. Different functions contain different GET or POST requests that call the intended route, which can provide the response they are looking for. For instance, the function for the user to view their order history will request a JSON from the route `{BASE_URL}/users/{user_id}/orders`. If there is an error, an error message will be displayed to the user.

For a smoother programme flow, the front-end incorporates graphical UI elements, including rounded borders, banners with emojis, section dividers, and colour-coded messages (magenta for errors, cyan for information, green for successes). Emojis related to stores, food, and delivery are used for the choice options, banners, and separators for a more intuitive design. All UI standardisation functions are located in the UI.py file, which also includes a custom render_table() function that displays aligned tables of the same style for stores, menus, orders, and drivers. The print functions for displaying bold tables are located at the bottom of the client.py file, and these functions are called in other functions, such as browse_stores() and show_orders().

4.3 Frontend-Backend Interaction (Assign Driver POST flow)

Front-end Call

After seeing an unshipped order, the admin can assign an active driver to it. The front-end issues a POST request to this route and uses the driver_id that the admin has selected:

```
driver_id_input = input("\nEnter Driver ID to assign: ")
try:
    driver_id = int(driver_id_input)
except ValueError:
    error("Invalid Driver ID.")
    return

assign_resp = requests.post( url: f"{BASE_URL}/admin/orders/{order_to_manage}/assign", json={"driver_id": driver_id})
```

Back-end Handling

In the backend, when the POST request hits /admin/orders/<order_id>/assign, the assign_driver(order_id) function is called:

```
@app.route( rule: "/admin/orders/<int:order_id>/assign", methods=["POST"])
def assign_driver(order_id):
    data = request.get_json()
    driver_id = data.get("driver_id")

    if driver_id is None:
        return jsonify({"success": False, "message": "Driver ID missing"}), 400

    conn = get_db_conn()
    cur = conn.cursor()
```

If driver_id is missing, the function will immediately return a failure message. If it exists, the function first establishes a connection to the SQLite database.

Then, it executes the following SELECT queries to check if some requirements are met. Namely, the order should be an existing UNSHIPPED order, and the driver should be an ACTIVE driver with a real existing driver_id. If any one of these requirements is violated, the connection to the database will close and the function will return error messages accordingly.

```
# Check if the order exists and is UNSHIPPED
cur.execute( __sql: "SELECT Status FROM Orders WHERE OrderID = ?", __parameters: (order_id,))
row1 = cur.fetchone()

if row1 is None:
    conn.close()
    return jsonify({"success": False, "message": "Order not found"}), 404
if row1["Status"] != "UNSHIPPED":
    conn.close()
    return jsonify({"success": False, "message": "Order is not unshipped"}), 400

# Check if the driver exists and is ACTIVE
cur.execute( __sql: "SELECT IsActive FROM Driver WHERE DriverID = ?", __parameters: (driver_id,))
row2 = cur.fetchone()

if row2 is None:
    conn.close()
    return jsonify({"success": False, "message": "Driver not found"}), 404

if row2["IsActive"] != 1:
    conn.close()
    return jsonify({"success": False, "message": "Driver is NOT active"}), 400
```

If all the requirements are met, the function will execute an UPDATE command to the Orders table, adding the driver_id chosen by the admin to the DriverID column for the specific order_id that the admin chose to manage. All changes made will be saved permanently to the database file before the connection closes. The function then returns a JSON response with a message:

```python
# Assign driver
cur.execute( __sql: """
    UPDATE Orders
    SET DriverID = ?
    WHERE OrderID = ?;
""",  __parameters: (driver_id, order_id))

conn.commit()
conn.close()
return jsonify({"success": True, "message": "Driver assigned successfully"})
```

CLI and Database Effect

Back in the front end, if the back-end function returns an error, the error message will be displayed to the user. If the back end successfully updates Orders with the driver_id, then a success message will be displayed.

```python
if not assign_resp.ok:
    error(f"Failed to assign driver: {assign_resp.text}")
else:
    success(assign_resp.json().get("message"))
```

CLI:

```
Enter Driver ID to assign: 201
✅Driver assigned successfully
```

Database update:

OrderID	OrderTi...	StoreID	UserID	Status	Deliver...	DriverID
1	2025-11-...	7	10001	UNSHIPP...	Teaching...	null

OrderID	OrderTi...	StoreID	UserID	Status	Deliver...	DriverID
1	2025-11-...	7	10001	UNSHIPP...	Teaching...	201

5 SUMMARY & HARVEST

In summary, this project achieved a functional system with a clear distinction between user and admin roles, over 20 role functions, a clean relational database, password hashing, and a standardised, aesthetically pleasing CLI user interface.

Although I initially misjudged how long it would take to create the functions and run a bug-free programme, I learned for the first time how to develop a project where the backend and frontend communicate using REST APIs. I also learned the importance of meticulous planning and the need for a user experience flowchart to prevent serious problems down the line, which I will apply to future projects.

Some future improvements I was not able to implement for this project yet are using a web UI, optimising driver assignment, and implementing a user rating system. These could further optimise the user experience.